Programming II - Part 2 Exam Conspect

Recursion, stacks, linked lists, trees, timing, SQL, sample test strategy, tiny microcontroller summary

Contents

1 Recursion 1

2 Stack Abstract Data Type 1

3 Linked Lists 2

4 Trees and Trie 3

5 Execution Time and Optimization 5

6 SQL and SQLite Basics 5

7 Sample Test Coding Strategy 6

8 Microcontroller Summary 7

9 High-Value Exam Answers 8

Recursion

Idea

Recursion means solving a problem by calling the same function on a smaller version of the same problem. It is natural for divide-and-conquer algorithms and self-similar data structures such as trees.

Theory. A recursive function must have: **base case** (when to stop) and **recursive step** (how to reduce the problem). Each call creates a new function call stack frame. Too many nested calls or a missing base case can cause stack overflow. Most recursive algorithms can be rewritten iteratively, but not always with the same clarity.

Factorial example

```
int Factorial(int n)
{
    if (n <= 1)        // base case: 0! = 1, 1! = 1
        return 1;
    return n * Factorial(n - 1);   // recursive step
}
```

Exam answer

Why does recursion need a base case? Without a base case, the function keeps calling itself forever. Every call uses stack memory, so the program will eventually crash with stack overflow.
Recursive vs iterative. Recursive code is often cleaner for trees/search/sorting, but it uses more stack memory. Iterative code can be more memory-safe for very deep input.

Stack Abstract Data Type

Idea

A stack is an abstract data type with **LIFO**: last in, first out. Only the top element is directly accessible, because apparently random access was too much freedom for one data structure.

Core operations. `Push` adds to top, `Pop` removes and returns top, `Peek` reads top without removing it. Optional operations include duplicate, swap, rotate, display. Common uses: function call stack, backtracking, syntax evaluation, postfix notation, iterative versions of recursive algorithms.

Important / danger

Stack overflow: push into full stack. **Stack underflow:** pop from empty stack. Array-based stacks need capacity checks. Linked-list stacks need allocation checks.

Small dynamic-array stack

```c
typedef struct {
    int *items;
    int size;      // number of elements currently stored
    int capacity;  // allocated length
} Stack;

int Push(Stack *s, int value)
{
    if (s->size == s->capacity) {
        int newCap = (s->capacity == 0) ? 4 : s->capacity * 2;
        int *tmp = realloc(s->items, sizeof(int) * newCap);
        if (tmp == NULL) return 0;
        s->items = tmp;
        s->capacity = newCap;
    }
    s->items[s->size++] = value;
    return 1;
}

int Pop(Stack *s, int *out)
{
    if (s->size == 0) return 0;     // underflow
    *out = s->items[--s->size];
    return 1;
}
```

Exam answer

Why is stack access fast? Because only the top element is used. Push/pop do not require searching through all stored data.

Array vs linked-list stack. Array stack uses sequential memory and may need resizing. Linked-list stack allocates one node at a time and avoids large continuous memory requirement, but each node has pointer overhead.

Linked Lists

Idea

A linked list is made of nodes allocated individually on the heap. Nodes are independent memory blocks connected by pointers. The program must keep at least `pHead`; often also `pTail` for faster append.

Theory. In a singly linked list, each node has `pNext`. The last node has `pNext == NULL`. A doubly linked list also has `pPrev`, so traversal works both directions and deletion is easier, but memory overhead and pointer setup are larger. Linked lists help with memory fragmentation because nodes do not need one large continuous block.

Important / danger

Adding/removing nodes means changing pointers in the correct order. If you lose the only pointer to a node, memory leaks. If you use a freed node, undefined behavior. If you forget to update `pHead`, the list becomes inaccessible, which is a very C way of saying “gone forever”.

Full singly linked list: declaration and helper

```c
typedef struct Node {
    int id;
    char *name;
    struct Node *pNext;
} Node;

static char *CopyString(const char *s)
{
    char *p = malloc(strlen(s) + 1);
    if (p != NULL) strcpy(p, s);
    return p;
}
```

Insert at front and at end

```c
int PushFront(Node **ppHead, int id, const char *name)
{
    Node *pNew = malloc(sizeof(Node));
    if (pNew == NULL) return 0;
    pNew->name = CopyString(name);
    if (pNew->name == NULL) { free(pNew); return 0; }

    pNew->id = id;
    pNew->pNext = *ppHead;
    *ppHead = pNew;
    return 1;
}

int Append(Node **ppHead, int id, const char *name)
{
    Node *pNew = malloc(sizeof(Node));
    if (pNew == NULL) return 0;
    pNew->name = CopyString(name);
    if (pNew->name == NULL) { free(pNew); return 0; }
    pNew->id = id;
    pNew->pNext = NULL;

    if (*ppHead == NULL) { *ppHead = pNew; return 1; }

    Node *pCur = *ppHead;
    while (pCur->pNext != NULL) pCur = pCur->pNext;
    pCur->pNext = pNew;
    return 1;
}
```

Print and free list

```c
void PrintList(const Node *pHead)
{
    for (const Node *p = pHead; p != NULL; p = p->pNext)
        printf("%d %s\n", p->id, p->name);
}

void FreeList(Node *pHead)
{
    while (pHead != NULL) {
        Node *pNext = pHead->pNext;
        free(pHead->name);
        free(pHead);
        pHead = pNext;
    }
}
```

Exam answer

Why use a double pointer for insertion at front? The function must modify the caller's `pHead`. Passing `Node *pHead` gives only a copy of the pointer. Passing `Node **ppHead` gives access to the original pointer variable.
Why can linked lists handle fragmented memory better? Each node is allocated separately, so the program does not need one large continuous memory block.

Trees and Trie

Idea

A tree is a hierarchical structure. Each element is a node. Trees are naturally recursive because every subtree is also a tree, which is convenient, suspiciously elegant, and therefore easy to misuse.

Terminology. Root = top node. Parent/child = direct relation. Leaf = node without children. Internal node = has child nodes. Degree = number of children. Depth = distance from root. Height = longest path from root to a leaf.

Binary search tree (BST). Each node has up to two children. In a BST, left subtree contains smaller values, right subtree contains larger values. If balanced, search/insert is about $O(\log n)$. If badly unbalanced, it degenerates to $O(n)$, basically a linked list wearing a tree costume.

Full BST: structure, create node, insert

```c
typedef struct TreeNode {
    int value;
    struct TreeNode *pLeft;
    struct TreeNode *pRight;
} TreeNode;

TreeNode *CreateNode(int value)
{
    TreeNode *n = malloc(sizeof(TreeNode));
    if (n == NULL) return NULL;
    n->value = value;
    n->pLeft = NULL;
    n->pRight = NULL;
    return n;
}

int Insert(TreeNode **ppRoot, int value)
{
    if (*ppRoot == NULL) {
        *ppRoot = CreateNode(value);
        return *ppRoot != NULL;
    }
    if (value < (*ppRoot)->value)
        return Insert(&(*ppRoot)->pLeft, value);
    if (value > (*ppRoot)->value)
        return Insert(&(*ppRoot)->pRight, value);
    return 1;  // duplicate ignored
}
```

BST search, traversal, free

```c
TreeNode *Search(TreeNode *root, int value)
{
    if (root == NULL || root->value == value) return root;
    if (value < root->value) return Search(root->pLeft, value);
    return Search(root->pRight, value);
}

void PrintInOrder(TreeNode *root)
{
    if (root == NULL) return;
    PrintInOrder(root->pLeft);
    printf("%d\n", root->value);
    PrintInOrder(root->pRight);
}

void FreeTree(TreeNode *root)
{
    if (root == NULL) return;
    FreeTree(root->pLeft);
    FreeTree(root->pRight);
    free(root);
}
```

Notes

Traversal orders: **pre-order** = node, left, right. **in-order** = left, node, right; gives sorted order in BST. **post-order** = left, right, node; useful for freeing because children are freed before parent.

Trie structure only

```c
#define ALPHA_LEN 26

typedef struct TrieNode {
    int isEnd;                    // true if string ends here
    struct TrieNode *child[ALPHA_LEN];
} TrieNode;
```

Exam answer

Why are tree functions often recursive? A tree node points to smaller trees: left and right subtrees. The same operation can be applied to each subtree until `NULL`.

Why is trie lookup fast for strings? It follows one character per level, so lookup is proportional to string length, not number of stored strings. Cost: memory usage is high because each node keeps many child pointers.

Execution Time and Optimization

Idea

Performance is measured, not guessed. Revolutionary concept, apparently. Timing helps compare algorithms, memory strategies, and compiler effects.

Shell `time`. `real` is wall-clock time. `user` is CPU time in user mode. `sys` is CPU time in kernel mode: files, dynamic memory, process calls, device/driver work. `real` is not necessarily `user + sys` because waiting, scheduling, and parallelism can change the relation.

Timing with clock()

```c
#include <time.h>

clock_t start = clock();
/* code to measure */
clock_t end = clock();
double sec = (double)(end - start) / CLOCKS_PER_SEC;
printf("Time: %.6f s\n", sec);
```

Exam answer

Why is doubling capacity better than reallocating by +1? With `n + 1`, reallocation may happen for every inserted element: many allocations and copies. With doubling, capacity grows 4, 8, 16, 32, ... so the number of reallocations is about $\log_2 n$.

What is thrashing? If memory demand exceeds available RAM, the OS swaps pages to/from disk. Runtime can become dramatically slower because disk is much slower than RAM.

SQL and SQLite Basics

Idea

A database is an organized collection of data. A DBMS manages storage, retrieval, integrity, security, and often processing. SQL is declarative: describe what result is needed, not exactly how to compute it.

Relational model. Tables contain rows. A **primary key** uniquely identifies a row and cannot be `NULL`. A **foreign key** stores a reference to another table's primary key. Relationships include one-to-one, one-to-many, zero-to-many, and many-to-many. Many-to-many usually needs an extra linking table.

Small SQL examples

```sql
-- Select columns and filter rows
SELECT fname, lname, income
FROM person
WHERE income > 20000
ORDER BY income DESC
LIMIT 5;

-- Join two related tables
SELECT person.fname, person.lname, cars.car_reg_num
FROM person
JOIN cars ON cars.owner_id = person.id;

-- Insert strings with single quotes
INSERT INTO person (fname, lname, income)
VALUES ('Mari', 'Maasikas', 0);
```

Tiny SQLite C API pattern

```c
sqlite3 *db = NULL;
sqlite3_stmt *stmt = NULL;

if (sqlite3_open("data.db", &db) != SQLITE_OK) return 1;

const char *sql = "SELECT fname, lname FROM person WHERE income > ?;";
if (sqlite3_prepare_v2(db, sql, -1, &stmt, NULL) != SQLITE_OK) return 1;

sqlite3_bind_int(stmt, 1, 20000);   // safer than building SQL strings
while (sqlite3_step(stmt) == SQLITE_ROW) {
    const unsigned char *fname = sqlite3_column_text(stmt, 0);
    const unsigned char *lname = sqlite3_column_text(stmt, 1);
    printf("%s %s\n", fname, lname);
}

sqlite3_finalize(stmt);
sqlite3_close(db);
```

Important / danger

Do not build SQL by concatenating user input into strings. Use prepared statements and binding functions. Otherwise SQL injection becomes possible, and then the database starts taking life advice from random text input.

Exam answer

DBMS vs database. Database is the stored data. DBMS is the software that manages databases and provides access.
Why use primary/foreign keys? They create reliable relationships between tables without duplicating all related data.

Sample Test Coding Strategy

Idea

The sample test expects a practical C program: read any-length file into a dynamically allocated struct array, split source/header, output results, format numbers/date/time, and free memory. Tiny task. Naturally it wants all the ways C can hurt you in one place.

Typical algorithm.

1. Define a struct matching one file row.
2. Open file with `fopen`; check `NULL`.
3. Read using `fscanf`; use buffers with width limits for text fields.
4. Store into dynamic struct array using capacity doubling.
5. Process statistics: sums, averages, counters, max/common category.
6. Print formatted output: dates, times, percentages with 2 decimal places.
7. Free every dynamically allocated field and the array.

Sample-test style struct and reading loop

```c
typedef struct {
    int date;          // ddmmyyyy as integer from file
    char *feeling;     // dynamically copied
    int running, walking, standing, sitting, laying;
} Day;

int AppendDay(Day **arr, int *count, int *cap, Day item)
{
    if (*count == *cap) {
        int newCap = (*cap == 0) ? 8 : (*cap * 2);
        Day *tmp = realloc(*arr, sizeof(Day) * newCap);
        if (tmp == NULL) return 0;
        *arr = tmp;
        *cap = newCap;
    }
    (*arr)[(*count)++] = item;
    return 1;
}
```

Reading any-length ordinary text file

```
int ReadDays(const char *fileName, Day **outArr, int *outCount)
{
    FILE *fp = fopen(fileName, "r");
    if (fp == NULL) return 0;

    Day *arr = NULL;
    int count = 0, cap = 0;
    int date, run, walk, stand, sit, lay;
    char feelingBuf[64];

    while (fscanf(fp, "%d %63s %d %d %d %d %d",
                  &date, feelingBuf, &run, &walk, &stand, &sit, &lay) == 7) {
        Day d;
        d.date = date;
        d.feeling = CopyString(feelingBuf);
        if (d.feeling == NULL) { fclose(fp); return 0; }
        d.running = run; d.walking = walk; d.standing = stand;
        d.sitting = sit; d.laying = lay;

        if (!AppendDay(&arr, &count, &cap, d)) {
            free(d.feeling);
            fclose(fp);
            return 0;
        }
    }
    fclose(fp);
    *outArr = arr;
    *outCount = count;
    return 1;
}
```

Processing and formatting patterns

```
static void PrintDate(int ddmmyyyy)
{
    int day = ddmmyyyy / 1000000;
    int month = (ddmmyyyy / 10000) % 100;
    int year = ddmmyyyy % 10000;
    printf("%02d.%02d.%04d", day, month, year);
}

static void PrintTime(int minutes)
{
    printf("%02d:%02d", minutes / 60, minutes % 60);
}

static double ActivityPercent(const Day *d)
{
    return d->running * 1.37 + d->walking * 0.59 + d->standing * 0.18;
}
```

Important / danger

If reading fails halfway after allocating strings, already allocated memory still must be freed. A good solution has a cleanup function and uses it on both normal exit and error exit.

Microcontroller Summary

Idea

Tiny theory only, since this is not expected to dominate the exam.

A microcontroller is a small integrated control unit with CPU core(s), memory, and programmable I/O. It is used instead of a PC because it is smaller, lower-power, cheaper, and produces less heat. GPIO pins can be configured as input/output using direction registers. A port is an array/group of hardware registers. ADC converts analog voltage into a digital number; DAC converts digital number into analog voltage. Precision depends on reference voltage and bit width. Microcontrollers often use Harvard architecture: program memory and data memory are separate. Basys MX3 uses PIC32MX370F512L, 80 MHz, 512 KB Flash, 128 KB SRAM.

Exam answer

Why not use a PC everywhere? Power consumption, heat, size, and cost. Microcontrollers are enough for simple control tasks and embedded systems.

High-Value Exam Answers

Exam answer

What causes memory leakage? Losing all pointers to allocated memory, returning early without freeing, overwriting a pointer before `free`, forgetting to free dynamic fields inside structs, or not freeing all nodes in a linked structure.

Why do we need buffers for input? Functions like `fscanf` need a temporary fixed-size array to receive text. The buffer protects the program only if width limits are used, for example `%63s` for `char buf[64]`. Without width limits, long input can overflow the buffer.

When can buffer overflow happen? When more characters are written than the array can store, usually due to unsafe `scanf("%s", buf)`, `strcpy` into too-small destination, or missing space for the terminating `'\0'`.

Why is `realloc` used with a temporary pointer? If `realloc` fails, it returns `NULL` and the old block is still valid. Assigning directly to the original pointer loses the old address and leaks memory.

Why use `sizeof(*ptr)` style? It follows the actual pointed type. If the pointer type changes, allocation size remains correct.

Why is post-order useful for freeing trees? Children are freed first, then the parent. If the parent were freed first, its child pointers could no longer be safely accessed.

Why are linked lists slower for searching than arrays? They do not support direct indexing. To find an element, traversal starts from the head and follows pointers one by one.

Why can a BST become slow? If inserted values are already sorted, the tree can become a chain. Then search/insert becomes $O(n)$ instead of $O(\log n)$.

Why use prepared SQL statements? They separate SQL structure from values, avoid SQL injection, and allow safe binding of values with correct types.